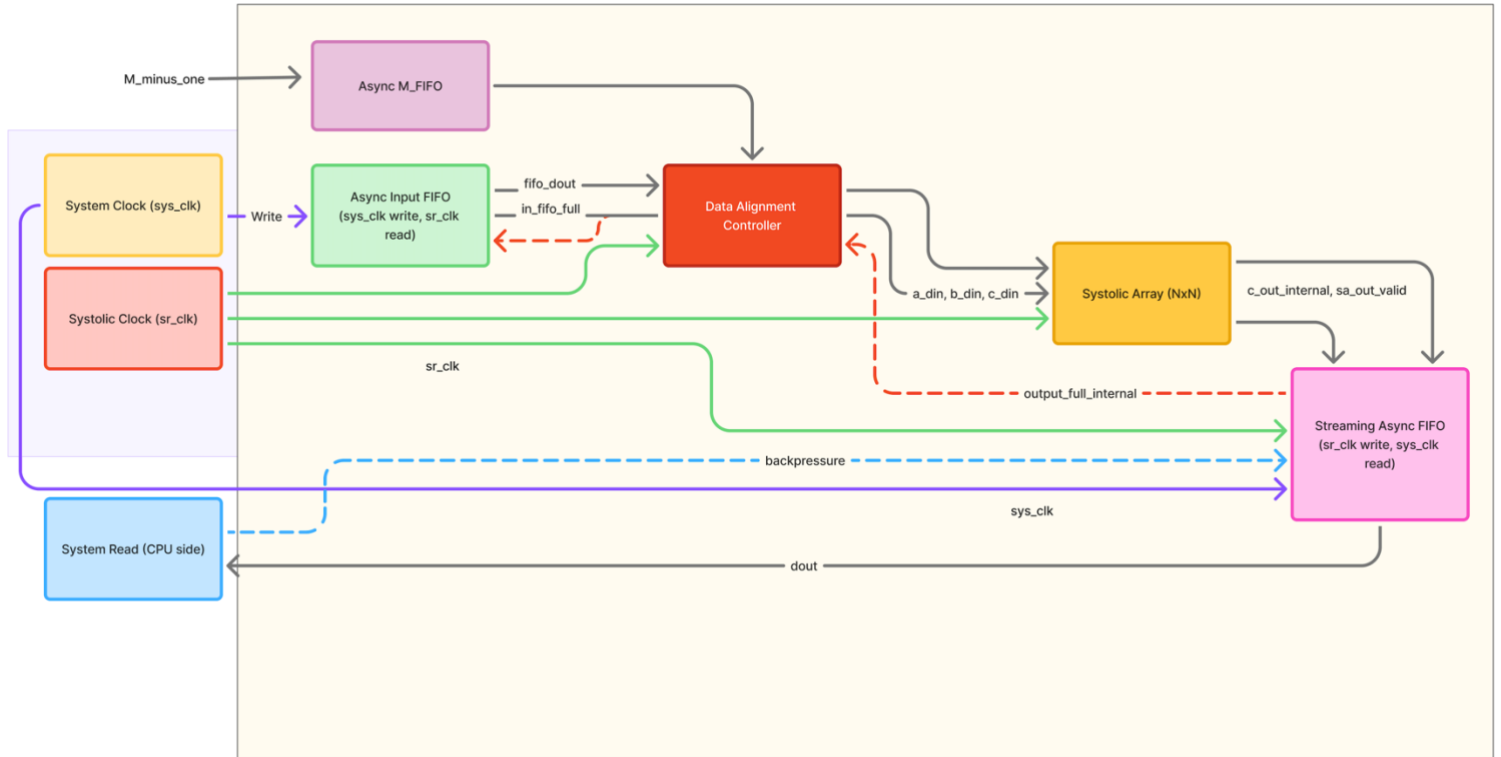


NxN Systolic Array Sub-System Design and UVM-TB Based Verification

Design

This is a cycle accurate RTL Design of the Systolic Array Sub-System written in System Verilog, most components are fully synthesizable, with exceptions to the Asynchronous FIFO, which is based on sv queues, instead of a Gray Code Encoded Asynchronous FIFO used in real-life designs.



The design is a sub-system for performing matrix multiplication using a systolic array architecture, with asynchronous FIFOs for clock domain crossing and data buffering. It consists of:

- **Input FIFO:** Buffers input data (columns of matrix A and rows of matrix B) written at sys_clk and read at sr_clk.
- **M FIFO:** Buffers the M-1 value (matrix dimension parameter) written at sys_clk and read at sr_clk, used by the data alignment controller.
- **Data Alignment Controller:** Manages skewed injection of data into the systolic array at sr_clk, handling backpressure from the output FIFO and using the M value for control.
- **Systolic Array:** Performs the actual NxM and MxN matrix multiplication at sr_clk, producing results as they become available.
- **Output FIFO:** Buffers the computed results written at sr_clk and read at sys_clk.
- **Instrumentation:** Provides debug monitoring and logging for various components, controlled by compile-time parameters.

The system supports configurable matrix sizes (N x M), data widths, and FIFO depths, with full and empty flags for flow control.

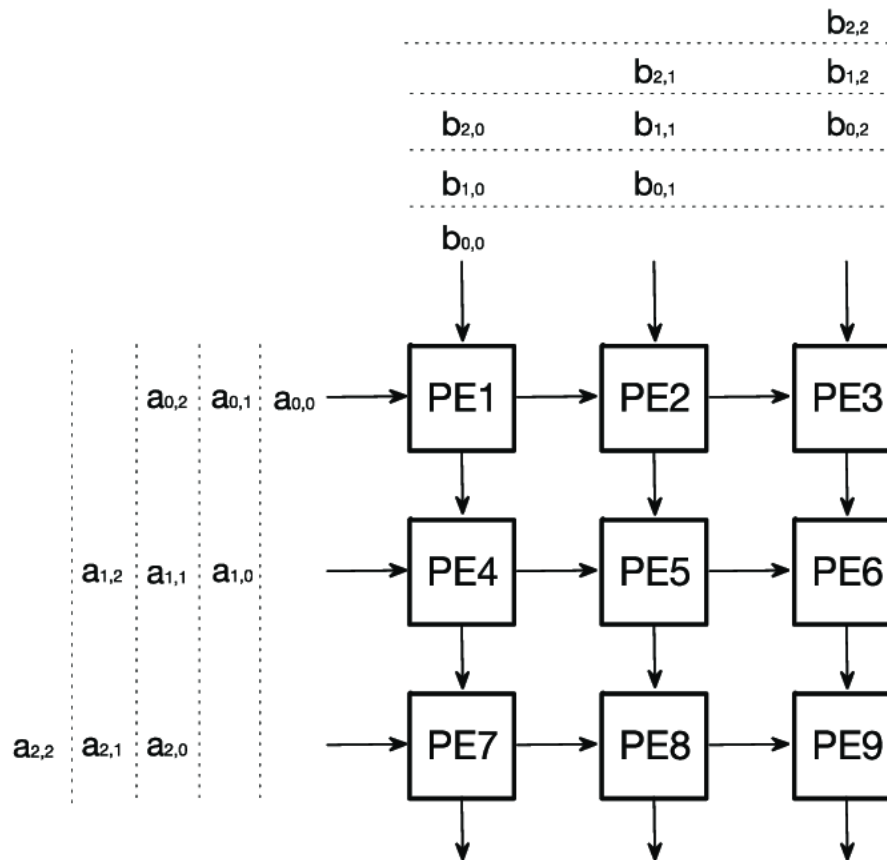
The design-related .sv files in this project are:

- **design.sv** – Sub-System Top Module
- **systolic_array.sv** – Systolic Array Module
- **pe.sv** – Processing Element
- **data_align_ctrl.sv** – Controller responsible for dequeuing Input fifo, and driving skewed data to the Systolic Array
- **async_fifo.sv** – SV Queues based Asynchronous FIFO
- **streaming_async_fifo.sv** – Systolic Array to Sub-System Output Queue Controller
- **instrumentation.sv** – It contains instrumentation code that shows detailed debug prints of pipeline stages

Disclaimer

As we will soon see below, that even though the UVM-Testbench supports different configurations for the Sub-system, the RTL Design however has existing bugs at the moment, which gives erroneous output often times when M and N are not equal, and some larger values of N, so the simulation run here focuses on (N=M=4 configuration). The bugs will be fixed in the future for a fully configurable design.

The Systolic Array Expects the controller to drive it's input data in the following format



Latency

1) Inside the array only (from in_valid to outputs)

- We assert in_valid on the last inject beat.
- The array then counts valid_count and asserts flush when $\text{valid_count} == 2*N - 1$.

Therefore:

- in_valid → flush (snapshot taken): $2*N - 2$ cycles
(since we load $\text{valid_count}=1$ on the in_valid cycle and hit $2N-1$ after $2N-2$ more cycles).
- We enter STREAM on the next cycle, so:
 - in_valid → first result row out: $2*N - 1$ cycles
 - We stream one row per cycle for N rows:
 - in_valid → last result row out: $(2*N - 1) + (N - 1) = 3*N - 2$ cycles

Summary (array-only):

First row latency = $2N - 1$ cycles after in_valid

Last row latency = $3N - 2$ cycles after in_valid

2) End-to-End including the start of data injection by aligner to all elements enqueued into the output fifo

The aligner injects for $N + M - 1$ cycles, with `in_valid` on the **last** inject cycle.

Let the **first inject beat** be time 0:

- **First inject** → `in_valid`: $N + M - 2$ cycles
- Add the array-only latencies above:

Therefore:

- **First inject** → **first result row**: $(N + M - 2) + (2N - 1) = 3N + M - 3$ cycles
- **First inject** → **last result row**: $(N + M - 2) + (3N - 2) = 4N + M - 4$ cycles

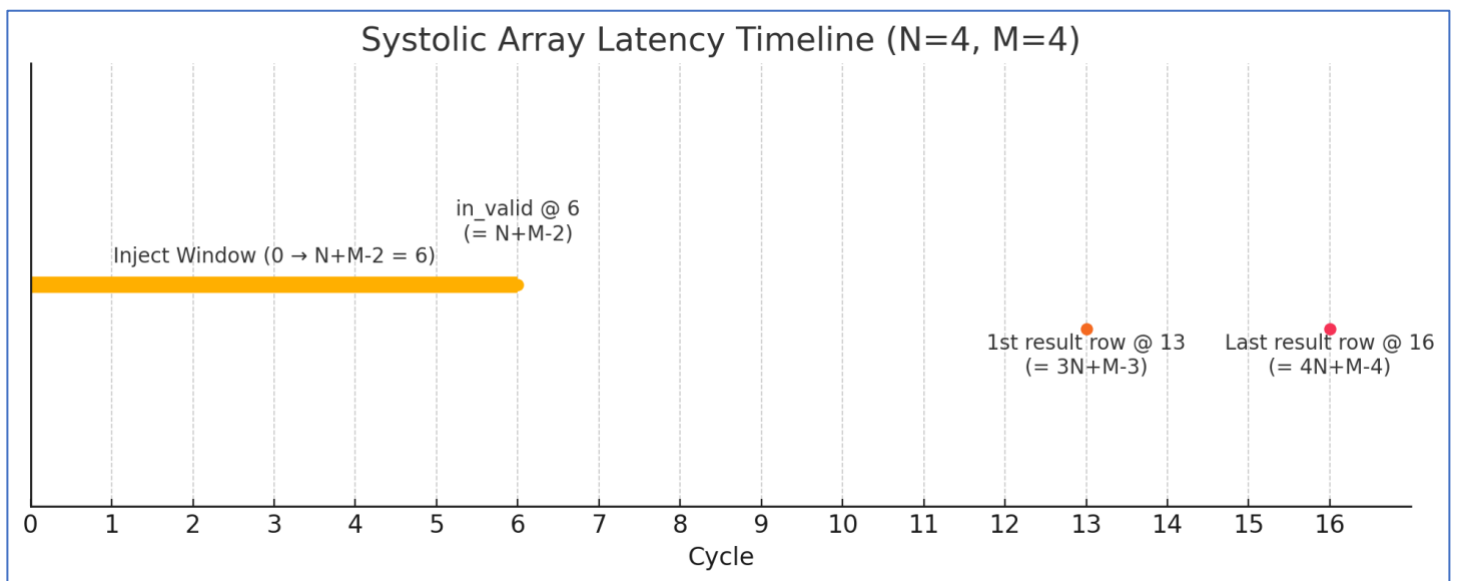
3) What can stretch it?

- **output_stall** in the array's STREAM state pauses the row streaming, so the **last-row** time increases by the number of stall cycles.
- Any stalls before `in_valid` (e.g., input backpressure) shift everything right but don't change the formulas relative to the reference event (`in_valid` or first inject).

Example:

For **N = 4**:

- Array-only:
 - First row after `in_valid`: $2 \cdot 4 - 1 = 7$ cycles
 - Last row after `in_valid`: $3 \cdot 4 - 2 = 10$ cycles
- With injection:
 - First inject → last row: $4N + M - 4 = 16 + M - 4 = M + 12$ cycles (no stalls)



NxN Systolic Array – Interface Description

Category	Signal	Direction	Width	Description
Parameters	DIN_WIDTH	Parameter	int	Specifies the data width (in bits) of a single A or B matrix element. The accumulator C elements are 2*DIN_WIDTH.
	N	Parameter	int	Defines the dimensions of the N x N square systolic array.
Global Signals	clk	Input	1	System clock. All operations are synchronous to the positive edge of this clock.
	rst_n	Input	1	Active-low asynchronous reset. Resets all internal registers and the output FSM.
Input Data	c_din	Input	[0:N-1] [2*DIN_WIDTH-1:0]	An array representing one row of initial C values. These values are loaded into the PEs of the <i>first row</i> (i=0) at the start of a computation (when the internal flush signal is high).
	a_din	Input	[0:N-1] [DIN_WIDTH-1:0]	An array of N input values for the A matrix. These are fed into the left edge (column 0) of the PE grid.
	b_din	Input	[0:N-1] [DIN_WIDTH-1:0]	An array of N input values for the B matrix. These are fed into the top edge (row 0) of the PE grid.
Input Control	in_valid	Input	1	A one-cycle pulse indicating last beat of a_din and b_din data. This pulse starts the internal latency counter, which runs for 2N-1 cycles to signal the computation is complete.
	output_stall	Input	1	Backpressure signal from a downstream module. When asserted high, this signal freezes the output FSM, holding the current c_dout row and pausing the row_idx counter.
Output Data	c_dout	Output	[0:N-1] [2*DIN_WIDTH-1:0]	The output data bus, which streams one full row of the NxN result matrix C per clock cycle (when not stalled).
Output Control	out_valid	Output	1	A single-cycle pulse that indicates the start of the output stream. It is asserted for one cycle, concurrent with the <i>first row</i> (row_idx=0) of the result matrix being valid on c_dout. The consumer is expected to accept N consecutive rows of data following this pulse.

Sub-System – Interface Description

Category	Signal	Direction	Width	Description
Parameters (Data)	DIN_WIDTH	Parameter	int	Data width (bits) of a single A or B matrix element.
	N	Parameter	int	Defines the dimensions of the NxN square systolic array.
	BUS_WIDTH	Parameter	int	Total width of the I/O data buses (din, dout). It is calculated as 2 * DIN_WIDTH * N.
Parameters (Debug)	DEBUG_ENABLE	Parameter	bit	Global master enable for all instrumentation and debug monitors.
	DEBUG_...	Parameter	bit	A set of parameters (DEBUG_ENABLE_INPUT_FIFO, DEBUG_ENABLE_SYSTOLIC, etc.) to granularly enable/disable specific instrumentation modules.
Global Signals	rst_n	Input	1	Active-low reset. This reset is applied to both clock domains.
	sys_clk	Input	1	System Clock: Clock for the external interface. Used for writing to the input FIFO (din, wr_fifo) and reading from the output FIFO (dout, rd_fifo).
	sr_clk	Input	1	Systolic Array Clock: Clock for the internal computation domain, including the data alignment controller and the systolic array itself.
Input Data & Control	M_minus_one	Input	[7:0]	The value (M-1) specifying the number of MAC operations for the NxM and MxN matrix multiplication. Written to an internal FIFO with wr_fifo.
	din	Input	[BUS_WIDTH-1:0]	Input data bus (domain: sys_clk). Contains one packed column of matrix A (N elements) and one packed row of matrix B (N elements).
	wr_fifo	Input	1	Write enable (domain: sys_clk). A high pulse writes the data on din and M_minus_one into their respective input FIFOs.
	rd_fifo	Input	1	Read enable (domain: sys_clk). A high pulse reads one entry (a full row of C) from the output FIFO onto dout.
Output Data & Status	in_fifo_full	Output	1	Input FIFO full status (domain: sys_clk). Asserts high when the input FIFO cannot accept new data.
	dout	Output	[BUS_WIDTH-1:0]	Output data bus (domain: sys_clk). Presents one full row of the result matrix C (\$N\$ elements, each 2*DIN_WIDTH bits wide) packed into a single BUS_WIDTH word.
	out_fifo_empty	Output	1	Output FIFO empty status (domain: sys_clk). Asserts high when no valid result data is available to be read.

Verification Using UVM Environment

Sub-System Verification Test Plan

1. Basic Functionality & Datapath

Goal: Verify that the core matrix multiplication logic $C_{i,j} = \sum_{k=0}^{M-1} A_{i,k} B_{k,j}$ correct under ideal (no-stall) conditions.

- **1.1. Test: TEST_SANITY_M_1**
 - **Description:** Send a single N×N matrix calculation (M=1).
 - **Constraints:** M_minus_one == 0. Clocks are synchronous (sys_clk == sr_clk). No stalls.
 - **Check:** Verify the single N×N result matrix C is bit-accurate.
- **1.2. Test: TEST_ACCUMULATION_M_GT_1**
 - **Description:** Send M=10 sets of A and B data to verify accumulation.
 - **Constraints:** M_minus_one == 9. Clocks are synchronous. No stalls.
 - **Check:** Verify the final accumulated N×N result matrix C is correct.
- **1.3. Test: TEST_DATA_ZERO**
 - **Description:** Send one matrix calculation where A or B is all zeros.
 - **Constraints:** M_minus_one == 0. All elements of A or B are zero.
 - **Check:** Verify the output C is all zeros (assuming the initial c_din load is also zero).
- **1.4. Test: TEST_DATA_IDENTITY**
 - **Description:** Send A=I (identity matrix) and any B.
 - **Constraints:** Randomize with Identity bit set
 - **Check:** Verify the output C equals B.

2. Backpressure & Stall Scenarios

Goal: Stress the FIFO-based handshaking and stall logic in the sr_clk domain.

- **2.1. Test: TEST_OUTPUT_STALL**
 - **Description:** Write a full matrix calculation (M=10) but never read the output.
 - **Constraints:** rd_fifo is *never* asserted. wr_fifo is pulsed INPUT_FIFO_DEPTH times.
 - **Check:**
 1. out_fifo_empty asserts low (showing data is ready).
 2. The output FIFO fills, causing output_full_internal to go high.
 3. The systolic array and aligner must stop consuming data.
 4. in_fifo_full must eventually assert high.
 5. The system remains stable with no data loss.
- **2.2. Test: TEST_INPUT_BURST**
 - **Description:** Fill the input FIFO as fast as possible, then drain the output FIFO.
 - **Constraints:** wr_fifo is asserted back-to-back M times. rd_fifo is only enabled after out_fifo_empty goes low.
 - **Check:** Verify the final C matrix is correct.
- **2.3. Test: TEST_INPUT_STALL (FIFO Empty)**
 - **Description:** Start a calculation (e.g., M=10) but only send 5 data packets.
 - **Constraints:** M_minus_one == 9. wr_fifo is asserted only 5 times.
 - **Check:**
 1. The data_align_ctrl reads the 5 packets and then idles (due to fifo_empty).
 2. The systolic array must not start its computation.
 3. out_fifo_empty must remain high. No partial result is produced.
- **2.4. Test: TEST_RANDOM_PACING**
 - **Description:** Run a long accumulation (M=100) with randomized input data and output draining.
 - **Constraints:** wr_fifo and rd_fifo are asserted with random delays.
 - **Check:** Verify the final C matrix is correct. This is the primary stress test for the handshake logic.

3. Asynchronous Clock Domain Crossing (CDC)

Goal: Verify data integrity across the sys_clk <-> sr_clk asynchronous boundary.

- **3.1. Test: TEST_CDC_SYNC (Baseline)**
 - **Description:** Run TEST_INPUT_BURST with synchronous clocks.
 - **Constraints:** sys_clk period == sr_clk period.
 - **Check:** Verify data is correct.

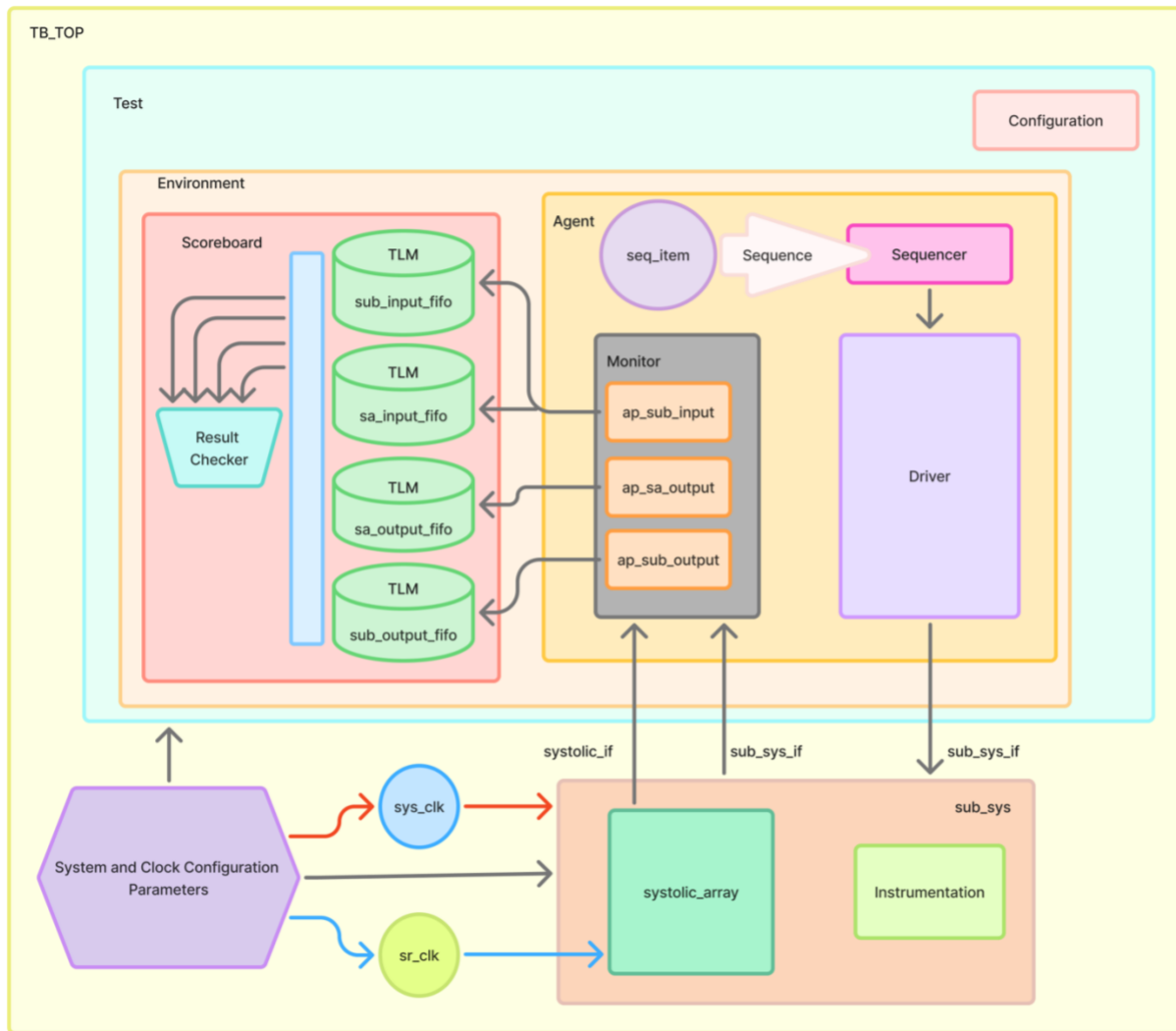
- **3.2. Test: TEST_CDC_FAST_TO_SLOW**
 - **Description:** The system writes data faster than the array can process it.
 - **Constraints:** sys_clk period < sr_clk period (e.g., sys_clk = 200MHz, sr_clk = 100MHz).
 - **Check:** in_fifo_full should assert frequently. The system must remain stable, and the final C matrix must be correct.
 - **3.3. Test: TEST_CDC_SLOW_TO_FAST**
 - **Description:** The system processes data faster than it can be written.
 - **Constraints:** sys_clk period > sr_clk period (e.g., sys_clk = 100MHz, sr_clk = 200MHz).
 - **Check:** in_fifo_full should rarely assert. The final C matrix must be correct.
 - **3.4. Test: TEST_CDC_NON_INTEGER**
 - **Description:** Use clock periods with a non-integer ratio to stress FIFO pointers.
 - **Constraints:** sys_clk and sr_clk have non-integer/prime-ratio periods (e.g., 100MHz vs 133MHz).
 - **Check:** Run a very long test (M=1000) and verify correctness.
-

4. Corner Cases & Parameters

Goal: Hit the boundaries of the design parameters and test reset recovery.

- **4.1. Test: TEST_MAX_M**
 - **Description:** Run the longest possible accumulation.
 - **Constraints:** M_minus_one == 255 (i.e., M=256).
 - **Check:** Verify correctness, ensuring no internal counters (like in data_align_ctrl) have overflowed.
- **4.2. Test: TEST_RESET_DURING_WRITE**
 - **Description:** Test reset recovery while writing to the input FIFO.
 - **Constraints:** Randomly assert rst_n (for 10+ cycles) while wr_fifo is being asserted.
 - **Check:** After reset, all FIFOs are empty, out_fifo_empty is high, and the system is ready for a new calculation.
- **4.3. Test: TEST_RESET_DURING_READ**
 - **Description:** Test reset recovery while data is being streamed out.
 - **Constraints:** Randomly assert rst_n while data is being read (rd_fifo is high, out_fifo_empty is low).
 - **Check:** System returns to a clean idle state.

UVM TB Architecture



Two-level Verification Approach, where it simultaneously checks both the top-level I/O and the internal systolic array interface.

1. Testbench Architecture (env, agent, test)

- **tb_top (Static):** This module instantiates the sub_sys DUT and the two virtual interfaces (sub_sys_if and systolic_if). It generates the clocks (sys_clk, sr_clk), handles the initial reset, and kicks off the UVM test via run_test("test").
- **test (Base Test):** This class (test) creates the env and a single configuration object (tb_config). It uses uvm_config_db to pass the configuration down to all other components. It then starts the main seq on the sequencer.
- **env (Environment):** The env class builds the two main UVM components: the agent (which handles stimulus and monitoring) and the scoreboard (which handles checking).
- **agent:** This is a standard UVM agent containing:
 - driver: Drives transactions to the DUT.
 - monitor: Observes transactions from the DUT.
 - sequencer: Manages the flow of sequence items to the driver.

2. Interfaces (Black-Box and White-Box)

This is the key design feature of this testbench.

1. sub_sys_if (Black-Box Interface):

- **Connection:** Plugs directly into the I/O ports of the sub_sys DUT.
- **Purpose:** Used for all standard UVM operations:
 - The driver uses it to send din, wr_fifo, rd_fifo, and M_minus_one.
 - The monitor uses it to watch in_fifo_full, out_fifo_empty, and all inputs/outputs.

- **Clocking:** It is clocked by sys_clk.
- 2. **systolic_if (White-Box Interface):**
 - **Connection:** This interface is **not** connected to the DUT's top-level ports. Instead, it uses **hierarchical references** (e.g., assign c_din = `DUT_HIER.u_sa.c_din;) to directly probe the internal signals between the data_align_ctrl and the systolic_array (u_sa).
 - **Purpose:** It gives the monitor "X-ray vision" to see the *internal* a_din, b_din, c_dout, and out_valid signals of the systolic array.
 - **Clocking:** It is clocked by sr_clk (via tb_if.sr_clk in tb_top), matching the systolic array's clock domain.

3. Stimulus Generation (seq_item, sequence, driver)

- **seq_item (Transaction):** This is a transaction object.
 - It randomizes the high-level matrices A and B and the M_minus_one value.
 - A post_randomize() function (pack_bus) "compiles" these 2D matrices into the 1D bus_din array, which is what the DUT actually expects.
 - It includes an over_under_flow_check() function to re-randomize if the generated data would create a math overflow, ensuring valid stimulus.
- **seq (Sequence):** A simple test sequence that creates and randomizes 15-20 seq_item transactions and sends them to the driver.
- **driver:** The driver's run_phase forks two parallel tasks:
 1. **drive_items:** Gets a seq_item, writes M_minus_one, and then loops M times, sending each bus_din packet while respecting the in_fifo_full backpressure signal.
 2. **rd_task:** This is a simple, continuous "drain". It constantly checks out_fifo_empty and asserts rd_fifo for one cycle whenever data is available.

4. Monitoring & Checking (monitor, scoreboard)

This is where the two-level verification happens.

- **monitor:** This component also forks parallel tasks to watch both interfaces.
 1. **sub_monitor_task (Black-Box):**
 - **Input:** Watches wr_fifo. It collects M+1 input packets, bundles them into a seq_item, and sends them to the scoreboard via ap_sub_input.
 - **Output:** Watches rd_fifo. It collects N output rows from dout, bundles them into a resp_item, and sends them to the scoreboard via ap_sub_output.
 2. **sa_monitor_task (White-Box):**
 - **Output:** Watches the systolic_if. It has a small state machine that waits for the *rising edge* of the *internal* out_valid signal.
 - It then captures the next N consecutive c_dout rows from the *internal* systolic array.
 - It bundles this into a systolic_resp_item and sends it to the scoreboard via ap_sa_output.
- **scoreboard:** The scoreboard is the central checker with two parallel checking tasks.
 1. **sub_checker (Top-Level Check):**
 - It gets one seq_item (from sub_input_fifo) and one resp_item (from sub_output_fifo).
 - It runs the golden model (sub_in.check_result(sub_out)) to verify the final $C=A \times B$ result is correct.
 - This verifies the **entire sub_sys design** from end to end.
 2. **sa_checker (Internal Check):**
 - It gets one seq_item (from sa_input_fifo) and one systolic_resp_item (from sa_output_fifo).
 - It runs a *separate* golden model (check_sa_result_from_sub) to verify the *internal* systolic array's output.
 - This **isolates the systolic_array logic** from the data_align_ctrl and FIFOs, helping to pinpoint bugs.

5. Configuration (tb_config)

Essential Knobs in tb_config

1. **agent_is_active**
 - **What it Controls:** The agent's mode (UVM_ACTIVE or UVM_PASSIVE).
 - **Why It's Essential:** It determines if the testbench will **drive stimulus** (Active) or just **passively monitor** the DUT (Passive).
2. **checker_mode**
 - **What it Controls:** Which scoreboard checkers are enabled.

- **Why It's Essential:** This is the key debug knob. It allows you to **isolate verification** to:
 - **Mode 1 (Both):** Verify the full sub-system and the internal systolic array.
 - **Mode 2 (Sub-system):** Verify only the top-level "black-box" sub-system i/o.
 - **Mode 3 (Systolic):** Verify only the internal "white-box" systolic array.

TB Files

Top-Level & Package Files

- **tb_top.sv:** The static, top-level Verilog module. It instantiates the sub_sys DUT, the two virtual interfaces (sub_sys_if, systolic_if), generates the clocks (sys_clk, sr_clk) and reset, and starts the UVM run_test().
- **sub_sys_tb_pkg.sv:** The main testbench package. It imports uvm_pkg and uses `include to compile all the UVM class files (like env, agent, seq_item, etc.) into a single package. It also defines the global testbench parameters (e.g., DIN_WIDTH, N, clock periods).
- **dut_hier_defines.sv:** A simple utility file that defines the `DUT_HIER macro. This macro provides the hierarchical path to the DUT, which is essential for the "white-box" systolic_if to probe internal signals.

Interface Files

- **sub_sys_if.sv:** The primary "**black-box**" interface. It connects directly to the top-level I/O ports of the sub_sys DUT. It's used by the driver and monitor to interact with the DUT's external sys_clk domain. It also contains basic SVA assertions.
- **systolic_if.sv:** The secondary "**white-box**" interface. It does *not* connect to the DUT's top-level. Instead, it uses the `DUT_HIER macro to "spy" on the internal signals between the data_align_ctrl and the systolic_array (u_sa). It's clocked by sr_clk and used only by the monitor.

UVM Environment Hierarchy

- **test.sv:** The uvm_test class. Its main job is to create the env and the tb_config object, set the configuration into the uvm_config_db, and start the main test sequence.
- **env.sv:** The top-level uvm_env class. It builds the agent (for stimulus/monitoring) and the scoreboard (for checking) and connects their analysis ports.
- **agent.sv:** The uvm_agent class. It builds the driver, monitor, and sequencer. It can be configured as UVM_ACTIVE (to drive stimulus) or UVM_PASSIVE (to only monitor).
- **config.sv:** (File contains class tb_config). Defines the central **configuration object** (tb_config). This object is passed to all components and acts as a "control panel" for the test, setting knobs like agent_is_active and the crucial checker_mode.

UVM Component Classes

- **driver.sv:** Drives stimulus to the DUT. It gets seq_item transactions from the sequencer, translates them into pin wiggles on the sub_sys_if, and correctly handles the in_fifo_full backpressure protocol.
- **monitor.sv:** The "eyes" of the testbench. It has two parallel tasks to watch both the sub_sys_if (black-box) and systolic_if (white-box). It reconstructs transactions from the pin activity and sends them to the scoreboard.
- **scoreboard.sv:** The "brain" of the testbench. It receives input and output transactions from the monitor and compares them. It contains two parallel checker tasks (sub_checker and sa_checker) to implement the dual-level (top-level and internal) verification logic.

UVM Transaction Classes

- **seq_item.sv:** Defines the main input transaction. It randomizes high-level A and B matrices and then, in post_randomize(), "compiles" them into the packed bus_din array that the driver will send.
- **resp_item.sv:** Defines the top-level output (response) transaction. It's a container for the final C[N][N] result matrix collected from the DUT's dout port.
- **systolic_resp_item.sv:** Defines the *internal* output transaction (a child of resp_item). It's a container for the C[N][N] result collected from the *internal* systolic_if.
- **sequence.sv:** (File contains class seq). A simple test sequence that creates, randomizes, and sends 15-20 seq_item transactions to the driver, starting the test.

Summary of the parameters and their default values defined in sub_sys_tb_pkg.sv:

Design & Geometry

- **DIN_WIDTH (8):** Data width of a single A or B element.
- **N (4):** The N×N dimension of the systolic array.
- **BUS_WIDTH (64):** Total width of the din/dout buses (2*DIN_WIDTH*N).

Clock & Timing

- **BASE_CLK_PERIOD (10.0 ns):** Base 100MHz reference period.
- **CLK_RATIO (0.6):** Ratio used to calculate the sys_clk period.
- **SYS_CLK_PERIOD (6.0 ns):** System (external I/O) clock period (166.67MHz).
- **SR_CLK_PERIOD (10.0 ns):** Systolic array (internal) clock period (100MHz).

DUT Instrumentation

- **DEBUG_ENABLE (0):** Global master switch for the DUT's internal debug messages.
- **DEBUG_ENABLE_INPUT_FIFO (1):** Enable debug for the input FIFO.
- **DEBUG_ENABLE_ALIGN_CTRL (1):** Enable debug for the data align controller.
- **DEBUG_ENABLE_SYSTOLIC (1):** Enable debug for the systolic array.
- **DEBUG_ENABLE_OUTPUT_FIFO (1):** Enable debug for the output FIFO.
- **DEBUG_ENABLE_MATRIX_RESULT (1):** Enable printing of the final matrix result from instrumentation.
- **DEBUG_ENABLE_SUMMARY (0):** Enable periodic summary prints from instrumentation.
- **DEBUG_ENABLE_DATA_TRACE (1):** Enable debug for A/B data flow.

Testbench Constraints (Localparams)

- **OP_MIN / OP_MAX:** Legal value range for a single A/B operand (e.g., -128 to 127).
- **RES_MIN / RES_MAX:** Legal value range for an accumulated C result (e.g., -32768 to 32767).

Monitoring and Scoreboard Results Checking

Analysis Ports (Defined in monitor)

These are the "broadcasters" that send out transactions.

1. **ap_sub_input**: Broadcasts the top-level input transaction (seq_item) seen on the sub_sys_if.
 2. **ap_sub_output**: Broadcasts the top-level output transaction (resp_item) seen on the sub_sys_if.
 3. **ap_sa_output**: Broadcasts the *internal* systolic array output (systolic_resp_item) seen on the systolic_if.
-

TLM Analysis FIFOs (Defined in scoreboard)

These are the "inboxes" that catch and queue the broadcasted transactions.

1. **sub_input_fifo**: Catches the top-level input (seq_item) from ap_sub_input.
2. **sub_output_fifo**: Catches the top-level output (resp_item) from ap_sub_output.
3. **sa_input_fifo**: Also catches the top-level input (seq_item) from ap_sub_input. (This FIFO is used by the sa_checker).
4. **sa_output_fifo**: Catches the internal systolic array output (systolic_resp_item) from ap_sa_output.

The scoreboard performs its checks in two parallel, independent tasks: **sub_checker** (for the top-level) and **sa_checker** (for the internal array).

Both tasks follow the same simple, self-synchronizing pattern:

1. **Wait for Input**: The task blocks on its input FIFO using .get() (e.g., sub_input_fifo.get(sub_in)).
2. **Wait for Output**: It then blocks on its corresponding output FIFO using .get() (e.g., sub_output_fifo.get(sub_out)).
3. **Compare**: Once it has *both* an input and an output transaction, it calls a comparison function (like sub_in.check_result(sub_out)). This function calculates the "golden" or expected C matrix and compares it, element by element, against the actual result from the DUT.

If the results match, it logs a UVM_INFO; if they mismatch, it flags a UVM_ERROR. Finally, the report_phase counts all UVM_ERRORS to declare a final **PASS** or **FAIL** for the test.

End Of Test

Summary of test's end phases, which are handled by your scbd (scoreboard).

1. extract_phase: Checking for Leftovers

- **Purpose**: To verify that all transactions were correctly processed and no data was "lost" or "unexpected."
- **How it works**:
 - This phase runs *after* the run_phase is complete.
 - It checks your four TLM FIFOs (sub_input_fifo, sub_output_fifo, sa_input_fifo, and sa_output_fifo) to see if any "leftover" transactions remain.
 - It uses a while loop with **try_get()** to non-blockingly check each FIFO.
 - If try_get() finds an item, it means a checker task (sub_checker or sa_checker) **failed to retrieve it**. This is a synchronization failure, and it logs a **UVM_ERROR**.
 - This catches bugs like the monitor sending 5 outputs but the scoreboard only expecting 4.

2. report_phase: Declaring the Final Verdict

- **Purpose**: To provide a single, clear **PASS** or **FAIL** message for the entire test.
- **How it works**:
 - This is the very last phase.
 - It gets the UVM report server and calls **svr.get_severity_count(UVM_ERROR)** and **svr.get_severity_count(UVM_FATAL)**.
 - It adds these to get a total error_count. This count includes **all** errors from the *entire test*:
 1. Data mismatches from the run_phase (e.g., sub_checker failed).
 2. Leftover item errors from the extract_phase.
 - It then prints the final verdict: if error_count == 0, it prints " OVERALL TEST PASSED "; otherwise, it prints "OVERALL TEST FAILED ".

Test-Results

EDA Playground: <https://www.edaplayground.com/x/Jk8s>

UVM_INFO testbench.sv(78) @ 0: reporter [TB_TOP] Asserting reset

=== Systolic Array Testbench Configuration ===

Time: 0.00 ns

Clock Settings:

System Clock (sys_clk):

Period : 6.00 ns

Frequency : 166.67 MHz

Systolic Array Clock (sr_clk):

Period : 10.00 ns

Frequency : 100.00 MHz

Clock Ratio : 0.60 (sr_clk:sys_clk)

=====

=== Starting Test at Time: 0.00 ns ===

UVM_INFO @ 0.00 ns: reporter [RNTST] Running test test...

UVM_INFO test.sv(34) @ 0.00 ns: uvm_test_top [test] Test Configuration:

=== Testbench Configuration ===

Agent Mode: UVM_ACTIVE

Coverage: Enabled

Checker Mode: 1

UVM_INFO /apps/vcsmx/vcs/U-2023.03-SP2/etc/uvm-1.2/src/base/uvm_root.svh(589) @ 0.00 ns: reporter [UVMTOP] UVM testbench topology:

Name	Type	Size	Value
uvm_test_top	test	-	@349
env_h	env	-	@367
agent_h	agent	-	@384
drv	driver	-	@540
rsp_port	uvm_analysis_port	-	@559
seq_item_port	uvm_seq_item_pull_port	-	@549
mon	monitor	-	@569
ap_sa_output	uvm_analysis_port	-	@598
ap_sub_input	uvm_analysis_port	-	@578
ap_sub_output	uvm_analysis_port	-	@588
seqr	uvm_sequencer	-	@403
rsp_export	uvm_analysis_export	-	@412
seq_item_export	uvm_seq_item_pull_imp	-	@530
arbitration_queue	array	0	-
lock_queue	array	0	-
num_last_reqs	integral	32	'd1
num_last_rsps	integral	32	'd1
scbd	scoreboard	-	@393
sa_input_fifo	uvm_tlm_analysis_fifo #(T)	-	@736
analysis_export	uvm_analysis_imp	-	@785
get_ap	uvm_analysis_port	-	@775
get_peek_export	uvm_get_peek_imp	-	@755
put_ap	uvm_analysis_port	-	@765
put_export	uvm_put_imp	-	@745
sa_output_fifo	uvm_tlm_analysis_fifo #(T)	-	@795
analysis_export	uvm_analysis_imp	-	@844
get_ap	uvm_analysis_port	-	@834
get_peek_export	uvm_get_peek_imp	-	@814
put_ap	uvm_analysis_port	-	@824
put_export	uvm_put_imp	-	@804
sub_input_fifo	uvm_tlm_analysis_fifo #(T)	-	@618
analysis_export	uvm_analysis_imp	-	@667
get_ap	uvm_analysis_port	-	@657

```
get_peek_export uvm_get_peek_imp      - @637
put_ap          uvm_analysis_port      - @647
put_export      uvm_put_imp            - @627
sub_output_fifo uvm_tlm_analysis_fifo #(T) - @677
analysis_export uvm_analysis_imp       - @726
get_ap          uvm_analysis_port      - @716
get_peek_export uvm_get_peek_imp      - @696
put_ap          uvm_analysis_port      - @706
put_export      uvm_put_imp            - @686
```

UVM_INFO driver.sv(30) @ 33.00 ns: uvm_test_top.env_h.agent_h.drv [driver] Reset De-Asserted....
UVM_INFO scoreboard.sv(67) @ 355.00 ns: uvm_test_top.env_h.scbd [SCOREBOARD] Systolic check passed
UVM_INFO scoreboard.sv(49) @ 369.00 ns: uvm_test_top.env_h.scbd [SCOREBOARD] Subsystem check passed
UVM_INFO scoreboard.sv(142) @ 369.00 ns: uvm_test_top.env_h.scbd [SCOREBOARD]
=== Subsystem Checker Details (Pass: 1) ===
Input Matrices:

=====

Sequence Item (M=4) | Stream: Continuous

=====

Matrix A [N x M]:

	1	0	0	0	
	0	1	0	0	
	0	0	1	0	
	0	0	0	1	

Matrix B [M x N]:

	91	14	90	110	
	-33	0	78	-31	
	0	-53	5	81	
	-37	57	2	53	

Packed bus_din values:

bus_din[0] = 0x6e5a0e5b00000001 | A_col: 1, 0, 0, 0 | B_row: 91, 14, 90, 110
bus_din[1] = 0xe14e00df00000100 | A_col: 0, 1, 0, 0 | B_row: -33, 0, 78, -31
bus_din[2] = 0x5105cb0000010000 | A_col: 0, 0, 1, 0 | B_row: 0, -53, 5, 81
bus_din[3] = 0x350239db01000000 | A_col: 0, 0, 0, 1 | B_row: -37, 57, 2, 53

Expected Output Matrix C:

	91	14	90	110	
	-33	0	78	-31	
	0	-53	5	81	
	-37	57	2	53	

DUT Output Matrix C:

	91	14	90	110	
	-33	0	78	-31	
	0	-53	5	81	
	-37	57	2	53	

UVM_INFO scoreboard.sv(67) @ 515.00 ns: uvm_test_top.env_h.scbd [SCOREBOARD] Systolic check passed
UVM_INFO scoreboard.sv(49) @ 531.00 ns: uvm_test_top.env_h.scbd [SCOREBOARD] Subsystem check passed
UVM_INFO scoreboard.sv(142) @ 531.00 ns: uvm_test_top.env_h.scbd [SCOREBOARD]
=== Subsystem Checker Details (Pass: 1) ===
Input Matrices:

=====

Sequence Item (M=4) | Stream: Continuous

=====

Matrix A [N x M]:

	-74	106	81	97	
	-83	100	48	70	
	-106	101	-26	5	
	32	88	45	46	

Matrix B [M x N]:

	-122	81	-32	61	
	88	127	25	0	
	25	76	52	73	
	49	-73	67	102	

Packed bus_din values:

bus_din[0] = 0x3de051862096adb6	A_col: -74, -83, -106, 32	B_row: -122, 81, -32, 61
bus_din[1] = 0x00197f585865646a	A_col: 106, 100, 101, 88	B_row: 88, 127, 25, 0
bus_din[2] = 0x49344c192de63051	A_col: 81, 48, -26, 45	B_row: 25, 76, 52, 73
bus_din[3] = 0x6643b7312e054661	A_col: 97, 70, 5, 46	B_row: 49, -73, 67, 102

Expected Output Matrix C:

	25134	6543	15729	11293	
	23556	4515	12342	5581	
	21415	1900	4900	-7854	
	7219	13830	6598	9929	

DUT Output Matrix C:

	25134	6543	15729	11293	
	23556	4515	12342	5581	
	21415	1900	4900	-7854	
	7219	13830	6598	9929	

Wave-form of the last 2 back-to-back operations showing pass for both sub-sys and sys arr check


```

UVM_INFO scoreboard.sv(180) @ 2773.00 ns: uvm_test_top.env_h.scbd [scoreboard] [PASS] All transactions in sub_output_fifo have been processed
UVM_INFO scoreboard.sv(188) @ 2773.00 ns: uvm_test_top.env_h.scbd [scoreboard] Checking systolic array FIFOs (mode 1 or 3)...
UVM_INFO scoreboard.sv(196) @ 2773.00 ns: uvm_test_top.env_h.scbd [scoreboard] [PASS] All transactions in sa_input_fifo have been processed
UVM_INFO scoreboard.sv(204) @ 2773.00 ns: uvm_test_top.env_h.scbd [scoreboard] [PASS] All transactions in sa_output_fifo have been processed
UVM_INFO scoreboard.sv(219) @ 2773.00 ns: uvm_test_top.env_h.scbd [scoreboard] --- Final Report Phase ---
UVM_INFO scoreboard.sv(223) @ 2773.00 ns: uvm_test_top.env_h.scbd [scoreboard] [Check]: PASS - No UVM_ERRORS or UVM_FATALs recorded.
UVM_INFO scoreboard.sv(229) @ 2773.00 ns: uvm_test_top.env_h.scbd [scoreboard] -----
UVM_INFO scoreboard.sv(231) @ 2773.00 ns: uvm_test_top.env_h.scbd [scoreboard] ** OVERALL TEST PASSED **
UVM_INFO scoreboard.sv(235) @ 2773.00 ns: uvm_test_top.env_h.scbd [scoreboard] -----
UVM_INFO /apps/vcs/mx/vcs/U-2023.03-SP2/etc/uvmm-1.2/src/base/uvmm_report_server.svh(904) @ 2773.00 ns: reporter [UVM/REPORT/SERVER]
--- UVM Report Summary ---

```

** Report counts by severity

```

UVM_INFO : 64
UVM_WARNING : 0
UVM_ERROR : 0
UVM_FATAL : 0

```

** Report counts by id

```

[RNTST] 1
[SCOREBOARD] 45
[TB_TOP] 1
[TEST_DONE] 1
[UVM/RELNOTES] 1
[UVMTOP] 1
[driver] 1
[scoreboard] 12
[test] 1

```

Basic Assertions for the Sub-System Interface

```

// =====
// ## Basic SVA Assertions
// =====

// --- Reset Assertions ---

// Check that during reset, the output FIFO is flagged as empty.
property p_reset_out_fifo_empty;
  !rst_n |-> out_fifo_empty;
endproperty
a_reset_out_fifo_empty: assert property (p_reset_out_fifo_empty) else
  `uvm_error("SVA", "out_fifo_empty must be high during reset")

// Check that during reset, the input FIFO is flagged as not full.
property p_reset_in_fifo_not_full;
  !rst_n |-> in_fifo_full;
endproperty
a_reset_in_fifo_not_full: assert property (p_reset_in_fifo_not_full) else
  `uvm_error("SVA", "in_fifo_full must be low during reset")

// --- FIFO Protocol Assertions ---

// Check that the driver never writes to the FIFO when it's full.
property p_no_write_when_full;
  disable iff (!rst_n) // Only check when not in reset
  !(wr_fifo && in_fifo_full);
endproperty
a_no_write_when_full: assert property (p_no_write_when_full) else
  `uvm_error("SVA", "Attempted to write (wr_fifo) to a full input FIFO (in_fifo_full)")

// Check that the driver never reads from the FIFO when it's empty.
property p_no_read_when_empty;
  disable iff (!rst_n)
  !(rd_fifo && out_fifo_empty);
endproperty
a_no_read_when_empty: assert property (p_no_read_when_empty) else
  `uvm_error("SVA", "Attempted to read (rd_fifo) from an empty output FIFO (out_fifo_empty)")

// --- Data Stability/Validity Assertions ---

// Check that the DUT status flags are never unknown (X or Z).
property p_status_flags_not_unknown;
  disable iff (!rst_n)
  !$isunknown({in_fifo_full, out_fifo_empty});
endproperty
a_status_flags_not_unknown: assert property (p_status_flags_not_unknown) else
  `uvm_error("SVA", "FIFO status flags (in_fifo_full, out_fifo_empty) are unknown")

// Check that if the output FIFO is not empty, its data is not unknown.
property p_dout_valid_when_not_empty;
  disable iff (!rst_n)
  !out_fifo_empty |-> !$isunknown(dout);
endproperty
a_dout_valid_when_not_empty: assert property (p_dout_valid_when_not_empty) else
  `uvm_error("SVA", "dout is unknown (X/Z) even though out_fifo_empty is low")

```

Endinterface

Instrumentation view of steps and transitions leading to first result


```
UVM_INFO driver.sv(30) @ 33.00 ns: uvm_test_top.env_h.agent_h.drv [driver] Reset De-Asserted....
[51.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=1/512
[51.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[1, 0, 0, 0] B_row=[91, 14, 90, 110]
[57.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=2/512
[57.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[0, 1, 0, 0] B_row=[-33, 0, 78, -31]
[63.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=3/512
[63.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[0, 0, 1, 0] B_row=[0, -53, 5, 81]
[65.00 ns] [SR_CLK] [ALIGN_CTRL] FIFO_RD=0 FIFO_EMPTY=0
[69.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=4/512
[69.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[0, 0, 0, 1] B_row=[-37, 57, 2, 53]
[75.00 ns] [SR_CLK] [ALIGN_CTRL] STATE: IDLE -> LOAD_REQ | LOAD_CNT=0 INJECT_CYCLE=0 CAPTURE_IDX=0 |
SA_COMPLETE=0 OUTPUT_STALL=0
[85.00 ns] [SR_CLK] [ALIGN_CTRL] STATE: LOAD_REQ -> LOAD_CAPTURE | LOAD_CNT=0 INJECT_CYCLE=0 CAPTURE_IDX=0 |
SA_COMPLETE=0 OUTPUT_STALL=0
[85.00 ns] [SR_CLK] [ALIGN_CTRL] FIFO_RD=1 FIFO_EMPTY=0
[85.00 ns] [SR_CLK] [ALIGN_DATA] FIFO_DATA (idx=0): A_col=[0, 0, 0, 0] B_row=[0, 0, 0, 0]
[87.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=4/512
[87.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[-74, -83, -106, 32] B_row=[-122, 81, -32, 61]
[93.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=5/512
[93.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[106, 100, 101, 88] B_row=[88, 127, 25, 0]
[95.00 ns] [SR_CLK] [ALIGN_CTRL] STATE: LOAD_CAPTURE -> INJECT | LOAD_CNT=0 INJECT_CYCLE=0 CAPTURE_IDX=0 |
SA_COMPLETE=0 OUTPUT_STALL=0
[95.00 ns] [SR_CLK] [ALIGN_DATA] INJECT cycle=0: A_din=[ - - - - ] B_din=[ - - - - ]
[99.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=6/512
[99.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[81, 48, -26, 45] B_row=[25, 76, 52, 73]
[105.00 ns] [SR_CLK] [ALIGN_CTRL] STATE: INJECT -> LOAD_CAPTURE | LOAD_CNT=1 INJECT_CYCLE=0 CAPTURE_IDX=0 |
SA_COMPLETE=0 OUTPUT_STALL=0
[105.00 ns] [SR_CLK] [ALIGN_CTRL] FIFO_RD=1 FIFO_EMPTY=0
[105.00 ns] [SR_CLK] [ALIGN_DATA] FIFO_DATA (idx=0): A_col=[1, 0, 0, 0] B_row=[91, 14, 90, 110]
[105.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=6/512
[105.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[97, 70, 5, 46] B_row=[49, -73, 67, 102]
[115.00 ns] [SR_CLK] [ALIGN_CTRL] STATE: LOAD_CAPTURE -> INJECT | LOAD_CNT=1 INJECT_CYCLE=0 CAPTURE_IDX=1 |
SA_COMPLETE=0 OUTPUT_STALL=0
[115.00 ns] [SR_CLK] [ALIGN_DATA] INJECT cycle=0: A_din=[ - - - - ] B_din=[ - - - - ]
[123.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=7/512
[123.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[0, 0, 92, 86] B_row=[9, -95, 64, -63]
[125.00 ns] [SR_CLK] [ALIGN_CTRL] STATE: INJECT -> LOAD_CAPTURE | LOAD_CNT=2 INJECT_CYCLE=0 CAPTURE_IDX=1 |
SA_COMPLETE=0 OUTPUT_STALL=0
[125.00 ns] [SR_CLK] [ALIGN_CTRL] FIFO_RD=1 FIFO_EMPTY=0
[125.00 ns] [SR_CLK] [ALIGN_DATA] FIFO_DATA (idx=1): A_col=[0, 1, 0, 0] B_row=[-33, 0, 78, -31]
[129.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=7/512
```

```
[129.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[-96, 11, 11, -22] B_row=[42, 116, 64, 25]
[135.00 ns] [SR_CLK] [ALIGN_CTRL] STATE: LOAD_CAPTURE -> INJECT | LOAD_CNT=2 INJECT_CYCLE=0 CAPTURE_IDX=2 |
SA_COMPLETE=0 OUTPUT_STALL=0
[135.00 ns] [SR_CLK] [ALIGN_DATA] INJECT cycle=0: A_din=[ - - - - ] B_din=[ - - - - ]
[135.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=8/512
[135.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[65, 49, -48, 76] B_row=[70, 0, 113, 30]
[141.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=9/512
[141.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[-24, 85, 62, 123] B_row=[36, -72, -17, 112]
[145.00 ns] [SR_CLK] [ALIGN_CTRL] STATE: INJECT -> LOAD_CAPTURE | LOAD_CNT=3 INJECT_CYCLE=0 CAPTURE_IDX=2 |
SA_COMPLETE=0 OUTPUT_STALL=0
[145.00 ns] [SR_CLK] [ALIGN_CTRL] FIFO_RD=1 FIFO_EMPTY=0
[145.00 ns] [SR_CLK] [ALIGN_DATA] FIFO_DATA (idx=2): A_col=[0, 0, 1, 0] B_row=[0, -53, 5, 81]
[155.00 ns] [SR_CLK] [ALIGN_CTRL] STATE: LOAD_CAPTURE -> INJECT | LOAD_CNT=3 INJECT_CYCLE=0 CAPTURE_IDX=3 |
SA_COMPLETE=0 OUTPUT_STALL=0
[155.00 ns] [SR_CLK] [ALIGN_DATA] INJECT cycle=0: A_din=[ - - - - ] B_din=[ - - - - ]
[159.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=9/512
[159.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[61, 100, 55, 16] B_row=[86, 116, 47, -25]
[165.00 ns] [SR_CLK] [ALIGN_CTRL] STATE: INJECT -> UNKNOWN | LOAD_CNT=0 INJECT_CYCLE=0 CAPTURE_IDX=3 |
SA_COMPLETE=0 OUTPUT_STALL=0
[165.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=10/512
[165.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[116, 38, 32, -60] B_row=[-8, 67, 93, -92]
[171.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=11/512
[171.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[112, -115, 34, 0] B_row=[18, -69, 44, -2]
[177.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=12/512
[177.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[33, -102, 59, 21] B_row=[-108, 96, -48, -65]
[195.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=13/512
[195.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[20, 59, -69, -128] B_row=[-20, -18, -47, -100]
[201.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=14/512
[201.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[-91, 31, 26, -70] B_row=[56, 55, -106, 106]
[207.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=15/512
[207.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[-60, 99, -36, 42] B_row=[106, 78, -49, 0]
[213.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=16/512
[213.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[-63, 12, 34, -5] B_row=[-17, -59, -114, 13]
[225.00 ns] [SR_CLK] [ALIGN_CTRL] *** in_valid PULSE *** (Data injection to SA started)
[225.00 ns] [SR_CLK] [ALIGN_DATA] *** DATA INJECTION TO SA *** (Full row/column injection)
A_din (horizontal flow) = [0, 0, 0, 1]
B_din (vertical flow) = [0, 0, 0, 53]
C_din (initial values) = [0, 0, 0, 0]
[231.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=17/512
[231.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[-82, -37, 101, -13] B_row=[-59, -19, 72, 50]
```

```
[235.00 ns] [SR_CLK] [ALIGN_CTRL] STATE: UNKNOWN -> LOAD_REQ | LOAD_CNT=0 INJECT_CYCLE=0 CAPTURE_IDX=3 |
SA_COMPLETE=1 OUTPUT_STALL=0

[237.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=18/512

[237.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[65, 8, -88, -81] B_row=[-112, -75, 91, 102]

[243.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=19/512

[243.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[28, -6, 105, 16] B_row=[-126, 44, 50, 117]

[245.00 ns] [SR_CLK] [ALIGN_CTRL] STATE: LOAD_REQ -> LOAD_CAPTURE | LOAD_CNT=0 INJECT_CYCLE=0 CAPTURE_IDX=0
| SA_COMPLETE=2 OUTPUT_STALL=0

[245.00 ns] [SR_CLK] [ALIGN_CTRL] FIFO_RD=1 FIFO_EMPTY=0

[245.00 ns] [SR_CLK] [ALIGN_DATA] FIFO_DATA (idx=0): A_col=[0, 0, 0, 1] B_row=[-37, 57, 2, 53]

[249.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=19/512

[249.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[13, -94, 9, -25] B_row=[-61, 115, 34, 82]

[255.00 ns] [SR_CLK] [ALIGN_CTRL] STATE: LOAD_CAPTURE -> INJECT | LOAD_CNT=0 INJECT_CYCLE=0 CAPTURE_IDX=0 |
SA_COMPLETE=3 OUTPUT_STALL=0

[255.00 ns] [SR_CLK] [ALIGN_DATA] INJECT cycle=0: A_din=[ - - - - ] B_din=[ - - - - ]

[265.00 ns] [SR_CLK] [ALIGN_CTRL] STATE: INJECT -> LOAD_CAPTURE | LOAD_CNT=1 INJECT_CYCLE=0 CAPTURE_IDX=0 |
SA_COMPLETE=4 OUTPUT_STALL=0

[265.00 ns] [SR_CLK] [ALIGN_CTRL] FIFO_RD=1 FIFO_EMPTY=0

[265.00 ns] [SR_CLK] [ALIGN_DATA] FIFO_DATA (idx=0): A_col=[-74, -83, -106, 32] B_row=[-122, 81, -32, 61]

[267.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=19/512

[267.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[60, -10, -83, 29] B_row=[10, -25, 12, 55]

[273.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=20/512

[273.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[-25, -74, 60, 114] B_row=[112, -10, 118, 74]

[275.00 ns] [SR_CLK] [ALIGN_CTRL] STATE: LOAD_CAPTURE -> INJECT | LOAD_CNT=1 INJECT_CYCLE=0 CAPTURE_IDX=1 |
SA_COMPLETE=5 OUTPUT_STALL=0

[275.00 ns] [SR_CLK] [ALIGN_DATA] INJECT cycle=0: A_din=[ - - - - ] B_din=[ - - - - ]

[279.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=21/512

[279.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[11, -36, 106, 21] B_row=[108, 96, 39, 115]

[285.00 ns] [SR_CLK] [ALIGN_CTRL] STATE: INJECT -> LOAD_CAPTURE | LOAD_CNT=2 INJECT_CYCLE=0 CAPTURE_IDX=1 |
SA_COMPLETE=6 OUTPUT_STALL=0

[285.00 ns] [SR_CLK] [ALIGN_CTRL] FIFO_RD=1 FIFO_EMPTY=0

[285.00 ns] [SR_CLK] [ALIGN_DATA] FIFO_DATA (idx=1): A_col=[106, 100, 101, 88] B_row=[88, 127, 25, 0]

[285.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=21/512

[285.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[15, 30, -52, 0] B_row=[-105, 121, 123, -115]

[295.00 ns] [SR_CLK] [ALIGN_CTRL] STATE: LOAD_CAPTURE -> INJECT | LOAD_CNT=2 INJECT_CYCLE=0 CAPTURE_IDX=2 |
SA_COMPLETE=7 OUTPUT_STALL=0

[295.00 ns] [SR_CLK] [ALIGN_DATA] INJECT cycle=0: A_din=[ - - - - ] B_din=[ - - - - ]

[295.00 ns] [SR_CLK] [SYSTOLIC_ARRAY] *** FLUSH EVENT *** (Matrix computation complete, snapshot taken)

[295.00 ns] [SR_CLK] [SA_DATA] C matrix snapshot at flush:

C_dout=[0, 0, 0, 0]

[303.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=22/512
```

```
[303.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[60, -49, 113, -23] B_row=[-58, -122, 71, 72]
[305.00 ns] [SR_CLK] [ALIGN_CTRL] STATE: INJECT -> LOAD_CAPTURE | LOAD_CNT=3 INJECT_CYCLE=0 CAPTURE_IDX=2 |
SA_COMPLETE=0 OUTPUT_STALL=0
[305.00 ns] [SR_CLK] [ALIGN_CTRL] FIFO_RD=1 FIFO_EMPTY=0
[305.00 ns] [SR_CLK] [ALIGN_DATA] FIFO_DATA (idx=2): A_col=[81, 48, -26, 45] B_row=[25, 76, 52, 73]
[305.00 ns] [SR_CLK] [SYSTOLIC_ARRAY] STATE: IDLE -> STREAM | ROW_IDX=0 VALID_CNT=0 | OUTPUT_STALL=0
[305.00 ns] [SR_CLK] [SA_DATA] Streaming row 0: C_dout=[0, 0, 0, 0]
[309.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=22/512
[309.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[-97, -15, 22, 121] B_row=[-17, -88, 85, 115]
[315.00 ns] [SR_CLK] [ALIGN_CTRL] STATE: LOAD_CAPTURE -> INJECT | LOAD_CNT=3 INJECT_CYCLE=0 CAPTURE_IDX=3 |
SA_COMPLETE=0 OUTPUT_STALL=0
[315.00 ns] [SR_CLK] [ALIGN_DATA] INJECT cycle=0: A_din=[ - - - - ] B_din=[ - - - - ]
[315.00 ns] [SR_CLK] [SA_DATA] Streaming row 1: C_dout=[91, 14, 90, 110]
[315.00 ns] [SR_CLK] [SYSTOLIC_ARRAY] *** out_valid PULSE *** (Streaming started)
[315.00 ns] [SR_CLK] [SA_DATA] Streaming results: C_dout=[91, 14, 90, 110]
[315.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=23/512
[315.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[0, -62, -128, 101] B_row=[74, 42, 69, 6]
[321.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=24/512
[321.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[-59, 55, 73, 57] B_row=[-76, 45, 72, -99]
[325.00 ns] [SR_CLK] [ALIGN_CTRL] STATE: INJECT -> UNKNOWN | LOAD_CNT=0 INJECT_CYCLE=0 CAPTURE_IDX=3 |
SA_COMPLETE=0 OUTPUT_STALL=0
[325.00 ns] [SR_CLK] [SA_DATA] Streaming row 2: C_dout=[-33, 0, 78, -31]
[325.00 ns] [SR_CLK] [OUTPUT_FIFO_WR] STATE: IDLE -> STREAMING | STREAM_CNT=0/4 | FULL=0 WR_EN=1
[325.00 ns] [SR_CLK] [OUTPUT_FIFO_WR] WR=1 FULL=0 EMPTY=0
[333.00 ns] [SYS_CLK] [OUTPUT_FIFO_RD] RD=1 EMPTY=0
[333.00 ns] [SYS_CLK] [MATRIX_RESULT] Row data: [0, 0, 0, 0]
[335.00 ns] [SR_CLK] [SA_DATA] Streaming row 3: C_dout=[0, -53, 5, 81]
[335.00 ns] [SR_CLK] [OUTPUT_FIFO_WR] WR=1 FULL=0 EMPTY=1
[339.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=25/512
[339.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[-125, -108, 113, 120] B_row=[-39, 0, 106, 115]
[345.00 ns] [SR_CLK] [SYSTOLIC_ARRAY] STATE: STREAM -> IDLE | ROW_IDX=0 VALID_CNT=0 | OUTPUT_STALL=0
[345.00 ns] [SR_CLK] [OUTPUT_FIFO_WR] WR=1 FULL=0 EMPTY=1
[345.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=26/512
[345.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[27, 2, -67, 44] B_row=[118, 48, 84, 115]
[345.00 ns] [SYS_CLK] [OUTPUT_FIFO_RD] RD=1 EMPTY=0
[345.00 ns] [SYS_CLK] [MATRIX_RESULT] Row data: [-9728, 28672, 23040, 30208]
[351.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=27/512
[351.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[22, -62, -25, -37] B_row=[-82, 74, -27, -2]
[355.00 ns] [SR_CLK] [OUTPUT_FIFO_WR] WR=1 FULL=0 EMPTY=1
UVM_INFO scoreboard.sv(67) @ 355.00 ns: uvm_test_top.env_h.scdb [SCOREBOARD] Systolic check passed
[357.00 ns] [SYS_CLK] [INPUT_FIFO] WR=1 FULL=0 EMPTY=1 DEPTH=28/512
```

```
[357.00 ns] [SYS_CLK] [INPUT_DATA] Writing to FIFO: A_col=[-98, 87, 48, -1] B_row=[-31, -125, 26, 84]
[357.00 ns] [SYS_CLK] [OUTPUT_FIFO_RD] RD=1 EMPTY=0
[357.00 ns] [SYS_CLK] [MATRIX_RESULT] Row data: [-1025, 0, 29184, -30721]
[365.00 ns] [SR_CLK] [OUTPUT_FIFO_WR] STATE: STREAMING -> IDLE | STREAM_CNT=0/4 | FULL=0 WR_EN=0
[369.00 ns] [SYS_CLK] [OUTPUT_FIFO_RD] RD=1 EMPTY=0
[369.00 ns] [SYS_CLK] [MATRIX_RESULT] Row data: [0, -11265, -24576, -30208]
UVM_INFO scoreboard.sv(49) @ 369.00 ns: uvm_test_top.env_h.scbd [SCOREBOARD] Subsystem check passed
UVM_INFO scoreboard.sv(142) @ 369.00 ns: uvm_test_top.env_h.scbd [SCOREBOARD]
=== Subsystem Checker Details (Pass: 1) ===
```

Input Matrices:

=====

Sequence Item (M=4) | Stream: Continuous

=====

Matrix A [N x M]:

```
| 1  0  0  0 |
| 0  1  0  0 |
| 0  0  1  0 |
| 0  0  0  1 |
```

Matrix B [M x N]:

```
| 91 14 90 110 |
| -33 0 78 -31 |
| 0 -53 5 81 |
| -37 57 2 53 |
```

Packed bus_din values:

```
bus_din[0] = 0x6e5a0e5b00000001 | A_col: 1, 0, 0, 0 | B_row: 91, 14, 90, 110
bus_din[1] = 0xe14e00df00000100 | A_col: 0, 1, 0, 0 | B_row: -33, 0, 78, -31
bus_din[2] = 0x5105cb0000010000 | A_col: 0, 0, 1, 0 | B_row: 0, -53, 5, 81
bus_din[3] = 0x350239db01000000 | A_col: 0, 0, 0, 1 | B_row: -37, 57, 2, 53
```

Expected Output Matrix C:

```
| 91 14 90 110 |
| -33 0 78 -31 |
```

```
| 0 -53 5 81 |  
| -37 57 2 53 |
```

DUT Output Matrix C:

```
-----  
| 91 14 90 110 |  
| -33 0 78 -31 |  
| 0 -53 5 81 |  
| -37 57 2 53 |
```

Future improvements

- Implement cross clock domain specific SVAs
- Implement functional SVAs
- Write Functional Coverage Module
- Write tests as laid out in the Verification Plan
- Fix existing controller bugs (Need to craft careful SVAs that monitors correct state transitions)
- Replace SV based Queue with Gray Coded Asynchronous FIFO
- Implement Advanced Architectural features like Double-Buffering, Multi-Tiling etc.